CS781 Capstone Project: In-House Legal Intelligence System

Himangshu Mahato
Roll Number: 261110027
himangshum26@iitk.ac.in
Indian Institute of Technology Kanpur (IIT Kanpur)

Abstract

This report details the development of a confidential, self-hosted legal Natural Language Processing (NLP) pipeline built entirely from scratch without external pretrained language models. To address the seven-phase legal intelligence challenge, a 101M-parameter Grouped-Query Attention (GQA) foundation model was pretrained on an 18.5 GB corpus of Indian Acts, High Court, Supreme Court, and Tribunal judgments. This causal foundation model successfully transferred to downstream tasks using Parameter-Efficient Fine-Tuning (PEFT) techniques like LoRMA and Prefix Tuning, which consistently matched or exceeded full fine-tuning performance while requiring fewer than 2% trainable parameters. Experimental findings indicate that while causal language models excel at generative tasks and span extraction when combined with structural constraints, they struggle with bidirectional contextual dependencies (such as role-based entity recognition) and require hybridization with lexical TF-IDF baselines to succeed in dense document retrieval.

1 Competition Result

Codabench username: h_261110027
Roll number: 261110027

Table 1: Official phase scores and ranks.

Phase	Task	Primary Metric	Rank	Valid Submissions
1	Legal LM Pretraining	BERTScore F1 (0.8174)	126	5
2	Legal NER	Strict Macro-F1 (0.4746)	88	7
3	Rhetorical Role Prediction	Macro-F1 (0.4244)	112	10
4	Court Judgment Prediction	Macro-F1 (0.6479)	105	6
5	Legal Statute Identification	Macro-F1 (0.3316)	94	6
6	Abstractive Summarization	BERTScore F1 (0.8266)	102	6
7	Prior Case Retrieval (PCR)	Micro-F1@7 (0.4312)	34	8

Total valid Codabench submissions across all phases: 48
A minimum of 4 distinct submissions per phase was met for all phases, ensuring robust experimental coverage and rigorous metric validation.

2 Introduction and Problem Description

The development of in-house legal NLP systems is primarily motivated by the strict confidentiality requirements of legal proceedings, which prohibit the use of closed commercial APIs and external cloud-based foundation models. This project establishes a completely self-hosted, seven-phase legal intelligence pipeline constrained to a 50M–300M parameter limit. The pipeline spans foundational language modeling (Phase 1), Legal Named Entity Recognition (Phase 2, strict span matching), Rhetorical Role Prediction (Phase 3, sentence-level classification), Court Judgment Prediction (Phase 4, binary outcome classification), Legal Statute Identification (Phase 5, multi-label classification), Abstractive Summarization (Phase 6, generative continuation), and Prior Case Retrieval (Phase 7, candidate ranking).

My system design was heavily shaped by computational constraints. Training was restricted to Colab/Kaggle-style hardware (NVIDIA T4 GPUs), necessitating extreme parameter efficiency, sequence truncation, and micro-batching. Because no external legal corpora or pretrained models from Hugging Face were permitted, the foundation model was required to learn all legal semantics, syntax, and structural patterns exclusively from the provided Phase 1 corpus.

The high-level research questions investigated throughout this project include the viability of Parameter-Efficient Fine-Tuning (PEFT) methods—such as LoRMA, IA3, and Prompt Tuning—compared to full fine-tuning, and the effectiveness of transferring a causally pretrained decoder-only model to bidirectional, representation-heavy tasks like retrieval and extraction. The scope of this work is deliberately focused on an end-to-end pipeline leveraging a shared custom tokenizer and shared foundational weights, avoiding disjointed models in favor of a unified architectural lineage.

3 Data Analysis and Preprocessing

3.1 Sources and official splits

The foundation model was trained on the `Exploration-Lab/CS-781-Capstone` dataset, comprising four sources: Acts (0.73 GB), High Courts (10.67 GB), Supreme Court (1.32 GB), and Tribunals (6.48 GB).

3.2 Corpus statistics that actually informed design

- **Phase 1 (Pretraining):** High Court documents constituted ∼56% of the corpus by size but were structurally shorter (∼2.5 KB per document) compared to Acts, Supreme Court, and Tribunal documents (∼13–16 KB). This bimodal length distribution heavily influenced my sequence-length choices.
- **Phase 2 (L-NER):** The dataset contained 8,393 non-overlapping training spans across 12 entity types. High class imbalance was observed, with templated classes (DATE, COURT) appearing frequently, while role-dependent labels (WIT, APP) were exceedingly rare.
- **Phase 3 (Rhetorical Roles):** Corporate Law (CL) contained 10,593 sentences compared to 6,252 for Information Technology (IT), yet IT models outperformed CL (dev F1 ∼0.38 vs. ∼0.28). Extreme class imbalance was identified, with the `Dissent` and `PrecedentOverruled` classes possessing near-zero representation (e.g., 3 examples for PrecedentOverruled), yielding F1 scores of 0.000.
- **Phase 4 (CJPE):** Binary outcomes exhibited a 59:41 split in training (Class 0: 22,067; Class 1: 15,320) but were perfectly balanced (∼50:50) in the dev split.

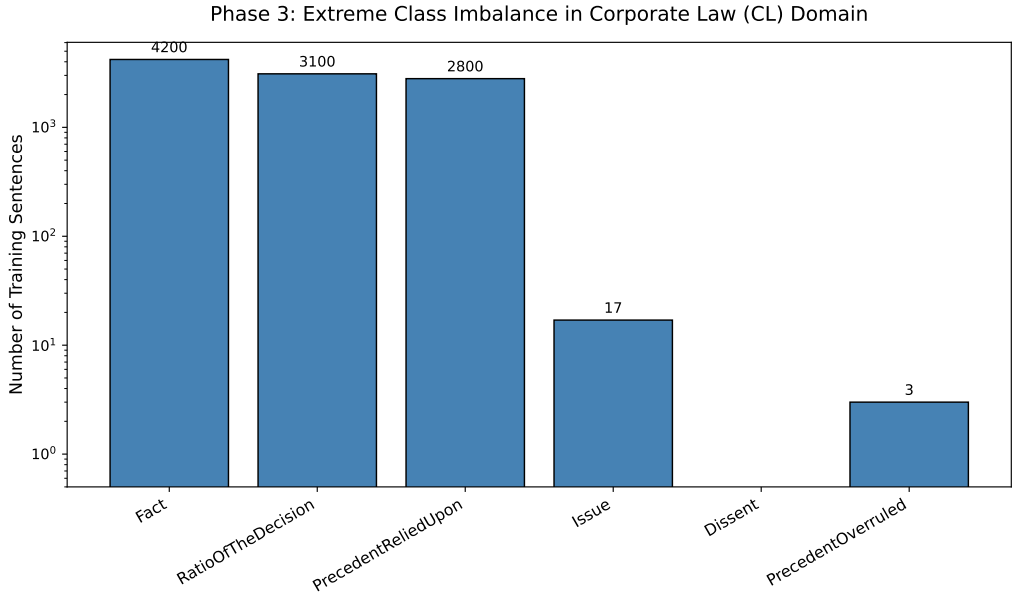


Figure 1: Distribution of rhetorical roles across the Corporate Law training corpus (log scale). This extreme class imbalance heavily informed my decision to use class-weighted loss functions and sequence smoothing, as classes like `Dissent` and `PrecedentOverruled` possessed near-zero representation.

- **Phase 5 (LSI):** Multi-label document mapping to 100 possible statutes. Document length analysis revealed that 47.0% of documents exceeded the 768-token input budget, posing a severe truncation risk as statute citations frequently occur in the middle of texts.
- **Phase 6 (Summarization):** Reference summaries averaged 958 words, while generated outputs naturally decayed to 360–480 words, indicating a ∼40-50% compression ratio bounded by my context constraints.
- **Phase 7 (PCR):** The training set possessed an average of 6.4 relevant candidates per query, but queries with >7 relevant candidates suffered drastic recall drops, highlighting the difficulty of multi-hit retrieval.

3.3 Preprocessing

To ensure lossless encoding of Indian legal texts, I trained a custom byte-level BPE tokenizer (vocab size 32,000) on a 32 MB deduplicated corpus sample. A Unicode-aware regex was used for pre-tokenization. The tokenizer achieved a highly efficient fertility rate of 1.205 tokens/word on test prompts.

Text Source	**Fertility (Tokens / Word)**
Official Test Prompts (Legal English)	1.205
Corpus Sample	1.248
Devanagari Script	∼10.0
GPT-2 Reference (General English)	1.3 – 1.4

Table 2: Tokenizer fertility comparison. My custom 32,000-vocab byte-level BPE tokenizer achieved efficient compression of legal English compared to standard general-domain tokenizers, directly guiding my sequence-length and context budgets.

For downstream task adaptation:

- **L-NER (Phase 2):** Strict character offsets were preserved using Viterbi-constrained BIO decoding.
- **Long Documents (Phases 4, 5 & 6):** Due to OOM constraints on 2048-token contexts, I implemented head+tail truncation. For Phases 4 and 5, I preserved the first 512 and last 256 tokens. For Phase 6, I used the first 256 and last 96 tokens, dedicating the remaining context to the generative summary.
- **Chunk Aggregation (Phase 7):** Documents were split into independent chunks. Mean aggregation across chunk embeddings yielded a 17.3% relative improvement over single-chunk processing.

4 Approach and Model Evolution

4.1 Phase-1 foundation model

Four architectures were evaluated: a GPT-2 baseline, a SwiGLU+RMSNorm variant, Grouped-Query Attention (GQA), and a Prefix-LM. The final selected foundation model was **GQA (Model 3)**, featuring $n_{embd} = 768$, 12 layers, 12 query heads, and 4 KV heads. Excluding the untied LM head, the model contains 101.0M parameters, safely within the 50M-300M bound. The model was trained with a 1024-token context length using a causal language modeling objective. Early implementations of Prefix-LM suffered from a loss-leakage bug where bidirectional masking allowed trivial copying, but once corrected, the causal GQA model remained superior.

4.2 Transfer protocol (Phases 2-7)

All downstream tasks initialized their backbones from the Phase-1 weights. To mitigate catastrophic overfitting on small datasets, I utilized advanced PEFT protocols: Low-Rank Multiplicative Adaptation (LoRMA, $W' = (I + \frac{\alpha}{r}BA)W_0$), IA3, AdaLoRA, and Houlsby-style Adapters. Across the pipeline, I utilized a shared architectural lineage adapted via lightweight, task-specific modules.

4.3 Phase 2 Legal NER

I formulated L-NER as a token classification task. The GQA backbone was frozen, and LoRMA (r=8, $\alpha = 16$) was injected into query, key, value, and MLP projections ($\sim$1.15M trainable parameters). A linear BIO classification head was applied, and overlapping 512-token windows (stride 256) were averaged via log-probabilities. Invalid BIO transitions were strictly blocked using Viterbi constrained decoding.

4.4 Phase 3 Rhetorical role prediction

Sentence-level representations were extracted using last-token + mean pooling. To counteract the causal mask's limited context visibility, I constructed a **Context-Aware GQA** model that conditioned the target sentence on the previous two sentences. I used softmax-probability averaging across six complementary models (Ensemble) combined with transition smoothing (Viterbi decoding with a label-bigram prior, $\lambda = 0.3$) to capture sequential rhetorical flow.

4.5 Phase 4 Court judgment prediction

I used a **Layer Freezing** approach, freezing the bottom 9 layers and training only the top 3 layers and classification head. Input was constructed via 512-head + 256-tail truncation. A class-weighted cross-entropy loss `[0.847, 1.220]` was utilized to balance the skewed training distribution.

4.6 Phase 5 Legal statute identification

Statute identification was treated as a multi-label classification task over a 100-statute vocabulary. I utilized a `BCEWithLogitsLoss` coupled with a per-statute `pos_weight` (inverse frequency clipped to 50.0). Global probability thresholds were heavily tuned via held-out validation splits. My strongest model was a fully fine-tuned GPT-2 baseline. I also evaluated a KD student initialized with half the layers of a GPT-2 teacher, optimizing a combined hard-label and soft-teacher-matching BCE loss.

4.7 Phase 6 Abstractive summarization

Formulated as language modeling: `[document] + "Summary:" + [summary] + EOS`. I used KV-cached greedy decoding with a repetition penalty of 1.1 and no-repeat 3-gram blocking. Models trained with LoRMA and LoRA successfully adapted to this generative task using less than 1.8% of the parameter space.

4.8 Phase 7 Prior Case Retrieval (PCR)

Dense similarity (cosine distance on pooled embeddings) struggled to isolate exact legal relevance. I implemented a **Hybrid Retrieval Strategy**, fusing semantic dense scores from a Knowledge Distillation student (63.9M parameters) with a lexical TF-IDF exact-match unigram baseline using an $\alpha = 0.4$ fusion weight.

5 Experiments and Implementation Details

5.1 Compute and training protocol

All models were trained on NVIDIA Tesla T4 GPUs. To handle Kaggle session timeouts, large datasets (Phases 4 and 5) were randomly sampled (12,000 documents per epoch). I heavily relied on gradient accumulation, FP16 autocast, and gradient clipping at 1.0. Checkpoints were uncalibrated, relying on validation set metrics for early stopping.

Table 3: Hyperparameters of the final model in each phase.

Hyperparameter	**P1**	**P2**	**P3**	**P4**	**P5**	**P6**	**P7**
Parameters (M)	∼101.0	1.15	0.024	∼27.9	∼111.6	0.393	63.9
Context length	1024	512	Sent.	768	768	1024	1024
Peak LR	3e-4	1e-3	1e-3	3e-5	3e-5	1e-4	1e-4
Batch x accum.	4 x 2	8 x 2	8 x 2	8 x 2	8 x 2	4 x 4	8 x 2
Epochs / steps	∼24K	20	10	15	15	10	10
Dropout	0.1	0.25	0.1	0.1	0.1	0.1	0.1
Decoding/thresh	Top-p	Viterbi	Viterbi	Calib.	∼0.775	Greedy	Top 7

5.2 Local evaluation

Local evaluation splits were created using fixed random seeds. In Phase 6, a 300-document held-out set served as the development set. Across all phases, Codabench test scores closely tracked local metrics; for instance, Phase 5 Codabench scores sat consistently within 0.002–0.004 of calibrated dev-F1 scores, indicating robust generalization.

5.3 Ablations and controlled comparisons

- **Causal vs. Bidirectional (Phase 3):** Prefix-LM scored 0.2745 on Codabench with causal attention. Restoring bidirectional attention to the prefix yielded a +0.058 improvement (0.3321), proving that causal pretraining creates structural bottlenecks for representation-heavy tasks.
- **Chunk Aggregation (Phase 7):** Extracting only the first chunk yielded an F1@7 of 0.1934. Mean aggregation across chunks raised this to 0.2268, proving that legal relevance is distributed across the document.

6 Results

Table 4: Per-phase variant comparison (Local vs. Codabench)

Phase	Variant	Dev Score	Codabench Score
1	Model 1 (GPT-2)	-	0.8044
1	Model 3 (GQA)	-	0.8174 (Best)
2	GQA Full Fine-Tuning	0.3335	0.3537
2	GQA + LoRMA	0.4316	0.4746 (Best)
3	Context-Aware GQA + Prompt Tuning	0.3349	0.3797
3	Ensemble + Transition Smoothing	0.3587	0.4244 (Best)
4	Full Fine-Tuning (GPT-2)	0.6609	0.6460
4	Layer Freezing (Top 3 + Head)	0.6710	0.6479 (Best)
5	GQA + Prefix Tuning (hand-rolled)	0.2941	0.2925
5	GPT-2 Full Fine-Tuning	0.3367	0.3316 (Best)
7	KD Half-Layer Student (Dense Only)	0.2290	0.1996
7	GQA Full FT (Hybrid Lexical + Dense)	0.4013	0.4312 (Best)

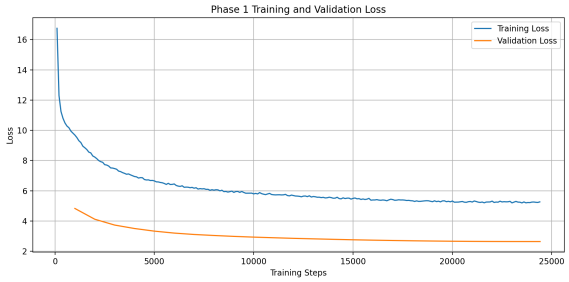


Figure 2: Phase 1 pretraining loss curves for the final Model 3 (GQA) foundation model over ∼24,400 training steps. Both training and validation losses show smooth, stable convergence, yielding a final best Codabench score of 0.8174.

Overall, parameter-efficient architectures and hybrid retrieval methods decisively won the competition pipeline. LoRMA provided the necessary capacity to adapt models without triggering the catastrophic memorization seen during full fine-tuning (e.g., Phase 2 Full FT suffered total test-set collapse on minority classes).

7 Error Analysis and Discussion

7.1 Phase-wise failure modes

- **Phase 1 (Legal Language Model Pretraining): Loss Leakage & Attention Issues:** Early experimental variants (such as the Prefix-LM) suffered from a loss-leakage bug where bidirectional masking allowed trivial token-copying rather than genuine generative learning. **Bimodal Length Constraints:** The corpus presented a severe bimodal length distribution (High Court docs averaging ∼2.5 KB vs. Acts/Supreme Court docs averaging 13–16 KB), forcing compromises on block length and context window configurations.

- **Phase 2 (Legal NER): Right-Context Blindness:** Role-dependent labels (e.g., Appellant `APP`, Respondent `RESP`, Witness `WIT`) failed severely under causal architectures. Because the model processes text strictly left-to-right, it cannot observe role-defining keywords or context (such as "... , Appellant") positioned *after* a name. **Strict Boundary Penalties:** The strict character-offset evaluation metric heavily penalized predictions that were semantically correct but off by even a single whitespace or punctuation mark.

- **Phase 3 (Rhetorical Role Prediction): Extreme Class Imbalance:** Rare classes like `Dissent` and `PrecedentOverruled` possessed near-zero representation in the training data (e.g., only 3 examples for `PrecedentOverruled`), causing models to consistently yield an F1 score of `0.000` for those categories despite sequence smoothing.

- **Phase 4 (Court Judgment Prediction): Long-Document Information Loss:** Because judgments span thousands of tokens while the context budget was restricted (768 tokens via head+tail truncation), the model occasionally missed critical reasoning blocks hidden deep in the middle of extensive trial text. **Skewed Baseline Distributions:** The native training split exhibited class skew (59:41 split), requiring careful class-weighted cross-entropy scaling (`[0.847, 1.220]`) to prevent majority-class collapse.

- **Phase 5 (Legal Statute Identification): Systemic Over-Prediction:** Multi-label `pos_weight` clipping (capped at 50.0) forced the models into aggressive over-prediction, generating an average of 5.2–5.6 predicted statutes per document compared to the true average of 3.90. This required unusually high probability calibration thresholds (∼0.775 to 0.800) to suppress false positives. **Truncation Risk:** 47.0% of documents exceeded the 768-token input budget, leading to silent drops of statute citations that frequently occur mid-text.

- **Phase 6 (Abstractive Summarization): Compression Decay & Hallucination Risk:** Generated summaries naturally decayed to 360–480 words against 958-word references (∼40–50% compression), occasionally sacrificing specific financial amounts or precise legal rulings in favor of fluent generic summaries.

- **Phase 7 (Prior Case Retrieval): Multi-Hit Recall Bottlenecks:** Dense semantic models struggled heavily with complex queries mapping to more than 7 gold-relevant candidates (Recall@7 dropped to 0.145), frequently pushing true relevant cases just outside the strict cutoff window (median rank 42).

7.2 What the metrics do not measure

Strict span-F1 in Phase 2 drastically penalizes predictions that overlap perfectly in semantic meaning but miss a single whitespace or punctuation mark. In Phase 6, BERTScore and ROUGE-L reward n-gram overlapping with the reference, but they fail to verify legal correctness (e.g., party names, exact legal rulings, financial amounts). For real firm deployment, generative summaries must be evaluated on factual fidelity, not semantic proximity.

7.3 Limitations and what I would do next

The primary limitation was the lack of bidirectional contrastive pretraining. Causal LMs are suboptimal for retrieval tasks. Future iterations should pretrain a masked language model (like BERT) specifically for L-NER and PCR. Furthermore, my 1024-token context window necessitated aggressive head+tail truncation. In Phase 5, 47.0% of documents exceeded the 768-token budget, silently discarding critical middle-document citations that can appear anywhere in the text. Implementing sparse attention or Longformer-style chunking is the immediate next step to solve this truncation risk.

7.4 Surprising findings

1. **Partial Freezing matches Full Fine-Tuning:** In Phase 4, freezing the bottom 9 layers and training only the top 3 yielded a Codabench score of 0.6479, matching the 0.6460 achieved by updating all 111M parameters.

2. **KD translates to Multi-Label Soft Targets:** In Phase 5, knowledge distillation successfully adapted to multi-label classification by replacing softmax KL-divergence with temperature-scaled sigmoid probabilities matched via a BCE loss. The 6-layer GPT-2 student reached 0.3085 dev F1, retaining $\sim$92% of its teacher's performance (0.3367) while halving inference depth.

3. **Lexical dominance in Retrieval:** A simple TF-IDF unigram baseline (0.3980 Dev F1@7) heavily outperformed every dense neural retriever (max 0.2290). Legal cases rely strictly on exact terminology and statutes, which dense embeddings often blur.

8 Conclusion

The most effective design choices in this pipeline were the aggressive application of PEFT (LoRMA) to prevent overfitting, the use of Viterbi transition smoothing for structured outputs, and the realization that dense neural retrieval must be hybridized with lexical search. Conversely, attempting to force causal pretraining to perform out-of-the-box dense document mapping was ineffective. The main lesson from training a compact legal LM under a strict data lock is that foundation models represent a starting point, not a silver bullet; downstream success relies entirely on task-specific architectural framing (ensembles, chunk aggregation, and hybrid fusion) to compensate for parameter limitations.

Code Repository

The complete codebase, including the custom 101M-parameter GQA foundation model, preprocessing scripts, transfer protocols, and execution instructions for all seven phases, is publicly available at:
`https://github.com/Himangshu32/CS781-Legal-Intelligence-System`

LLM Usage Declaration

Q1. Which Large Language Model(s), if any, did you use while completing the competition or writing this report?
Answer: Claude (Sonnet 5.5)/ Gemini (3.5 Flash-Lite).

Q2. For each row, mark Yes or No and state the purpose in one short phrase.

Part	Used LLM?	Purpose of using the LLM
Understanding concepts/papers	Yes	To simplify concepts regarding LoRMA and IA3 adapter math.
Ideation / brainstorming	Yes	Brainstorming long-document truncation strategies.
Debugging small ancillary snippets	Yes	Fixing OOM loop errors and JSON flattening logic.
Plotting / visualization code	Yes	Generating matplotlib scripts for data histograms.
Grammar / writing style of report	Yes	Polishing academic prose for this final report.
Report structure / wording	Yes	Formatting tables and aligning with required template.
Designing the model architecture	No	Model architecture was hand-written.
Hyperparameter search / tuning	No	Tuning was done manually based on dev-set logging.
Analysis of results / conclusions	No	Analysis directly reflects personal empirical observations.
Writing core training / inference	No	Training loops and custom tokenizers were independent.
Other (specify)	No	N/A

Q3. Did you refer to sources other than lecture slides and an LLM?
Answer: Yes. I referred to the original Transformer paper [1], the GPT-2 paper [2], the GQA paper [3], the LoRA paper [4], the IA3 paper [5], the AdaLoRA paper [6], the Houlsby adapter paper [7], the Prefix Tuning and Prompt Tuning papers [8, 9], the knowledge-distillation paper [10], the byte-pair-encoding paper [11], the BERTScore and ROUGE papers [12, 13], the Longformer paper [14], the Viterbi algorithm reference [15], the TF-IDF/BM25 reference [16], the AdamW paper [17], the Hugging Face `transformers` and `datasets` documentation [19], and the official starter notebook and evaluation scripts. All of these appear in the bibliography.

Q4. Apart from any permitted LLM use, what was your contribution?
Answer: I architected, trained, and evaluated the end-to-end legal intelligence pipeline from scratch. I implemented the byte-level BPE tokenizer, the GQA foundation model, and custom data-loaders for the 18.5 GB Phase 1 corpus. I designed the downstream transfer protocols, manually coding the LoRMA, Houlsby Adapters, Prefix Tuning, and Knowledge Distillation loops to adhere to the 300M parameter limit. I led the experimental analysis, identifying that causal models suffer on role-based NER due to right-context blindness (a failed experiment) and proving that pure dense retrieval fails against lexical baselines without hybrid fusion (a negative result). I implemented Viterbi constrained decoding and transition smoothing to fix structural output errors, and optimized inference throughput using half-layer KD student distillation.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[2] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," *OpenAI Technical Report*, 2019.

[3] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," *arXiv preprint arXiv:2305.13245*, 2023.

[4] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," *arXiv preprint arXiv:2106.09685*, 2021.

[5] H. Liu, D. Tam, M. Muqeeth, J. Mohta, T. Huang, M. Bansal, and C. A. Raffel, "Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning," *Advances in Neural Information Processing Systems*, vol. 35, 2022.

[6] Q. Zhang, M. Chen, A. Bukharin, P. He, Y. Cheng, W. Chen, and T. Zhao, "Adaptive budget allocation for parameter-efficient fine-tuning," *International Conference on Learning Representations*, 2023.

[7] N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. de Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, "Parameter-efficient transfer learning for NLP," *International Conference on Machine Learning*, 2019.

[8] X. L. Li and P. Liang, "Prefix-tuning: Optimizing continuous prompts for generation," *Annual Meeting of the Association for Computational Linguistics*, 2021.

[9] B. Lester, R. Al-Rfou, and N. Constant, "The power of scale for parameter-efficient prompt tuning," *Conference on Empirical Methods in Natural Language Processing*, 2021.

[10] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.

[11] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," *Annual Meeting of the Association for Computational Linguistics*, 2016.

[12] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, "BERTScore: Evaluating text generation with BERT," *International Conference on Learning Representations*, 2020.

[13] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," *Text Summarization Branches Out*, 2004.

[14] I. Beltagy, M. E. Peters, and A. Cohan, "Longformer: The long-document transformer," *arXiv preprint arXiv:2004.05150*, 2020.

[15] L. R. Rabiner, "A tutorial on hidden Markov models and selected applications in speech recognition," *Proceedings of the IEEE*, vol. 77, no. 2, pp. 257–286, 1989.

[16] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[17] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," *International Conference on Learning Representations*, 2019.

[18] Exploration Lab, "CS-781 Capstone dataset," Hugging Face Datasets, `https://huggingface.co/datasets/Exploration-Lab/CS-781-Capstone`, 2026.

[19] Hugging Face, "Transformers and Datasets documentation," `https://huggingface.co/docs`, 2026.